# NN&DL

# ?

S. Coniglio

Dipartimento di Scienze Economiche, UniBg

Sem. II

# Summary

1. SGD with momentum (SGDM)
2. Nesterov's accelerated (stochastic) gradient descent (NAG and SNAG)
3. Adam

# Notation in the next slides

In the next slides, we'll simplify our notation assuming that the parameters we are looking for are all contained in the vector $w$.

We will extend GD and SGD in various way.

For the ease of notation, we will assume that SGD is the basic method being extended, even though in the examples we will make we will always (for simplicity) assume a full-batch gradient.

# 1. SGD with momentum (SGDM)

# Momentum and SGD with momentum (SGDM)

A common modification to SGD is to add a **momentum** term to the update rule. The general idea is to average the gradient with the history of the past gradients in order to smoothen the trajectory we are taking.

The general rule

$$w^{(t+1)} := w^{(t)} - \alpha \sum_{i \in B^{(t)}} \nabla_w L_i(w^{(t)})$$

becomes the following one, with $\beta \in [0, 1)$:

$$\begin{aligned} w^{(t+1)} :=& w^{(t)} - \alpha m^{(t+1)} \\ m^{(t+1)} :=& \beta m^{(t)} + (1 - \beta) \sum_{i \in B^{(t)}} \nabla_w L_i(w^{(t)}) \end{aligned}$$

- $w^{(t+1)}$ is updated by making a step in the direction of $m^{(t+1)}$, rather than of the (batch) gradient
- $m^{(t+1)}$ is the **momentum** at iteration $t$: a convex combination with parameter $\beta \in [0, 1)$ between the current (batch) gradient $\sum_{i \in B^{(t)}} \nabla_w L_i(w^{(t)})$ and the previous direction $m^{(t)}$ used to update $w$.

Let's call $g^{(t)}$ the batch gradient at iteration $t$. Namely: $g^{(t)} := \sum_{i \in [k]} \nabla_w L_i(w^{(t)})$.

With this notation, $m^{(t+1)} := \beta m^{(t)} + (1-\beta) g^{(t)}$.

Let's see what the update rules lead to. Let's assume we start with $m^{(0)} = 0$ (this is reasonable: since we are yet to move, we have $0$ momentum).

$$
\begin{aligned}
m^{(1)} &:= (1-\beta) g^{(0)} \\
m^{(2)} &:= \beta m^{(1)} + (1-\beta) g^{(1)} \\
&= \beta(1-\beta) g^{(0)} + (1-\beta) g^{(1)} \\
m^{(3)} &:= \beta m^{(2)} + (1-\beta) g^{(2)} \\
&= \beta \left[ \beta(1-\beta) g^{(0)} + (1-\beta) g^{(1)} \right] + (1-\beta) g^{(2)} \\
&= \beta^2 (1-\beta) g^{(0)} + \beta(1-\beta) g^{(1)} + (1-\beta) g^{(2)} \\
&= (1-\beta) \left( \beta^2 g^{(0)} + \beta g^{(1)} + g^{(2)} \right).
\end{aligned}
$$

Warning: the exponents of $\beta$ are powers, not indices!!!

Generalizing:

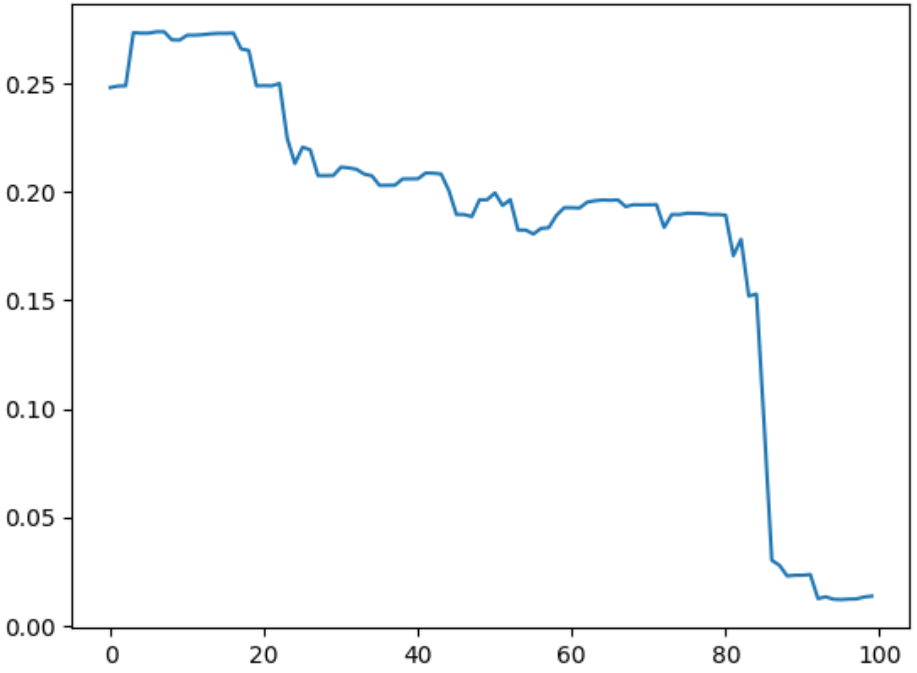
$$
\begin{aligned}
m^{(3)} &:= (1-\beta)\left(\beta^2 g^{(0)} + \beta g^{(1)} + g^{(2)}\right) \\
\vdots \qquad & \qquad\qquad \vdots \\
m^{(t+1)} &:= (1-\beta)\sum_{\tau=0}^{t} \beta^{t-\tau} g^{(\tau)}.
\end{aligned}
$$

This shows that, in SGD with momentum, rather than a step in the direction of the gradient, we make a step in a direction obtained by taking a weighted sum of all the previous gradients (i.e., we use an **accumulated history of past gradients**), where the weights get *exponentially* smaller as we move back in time.

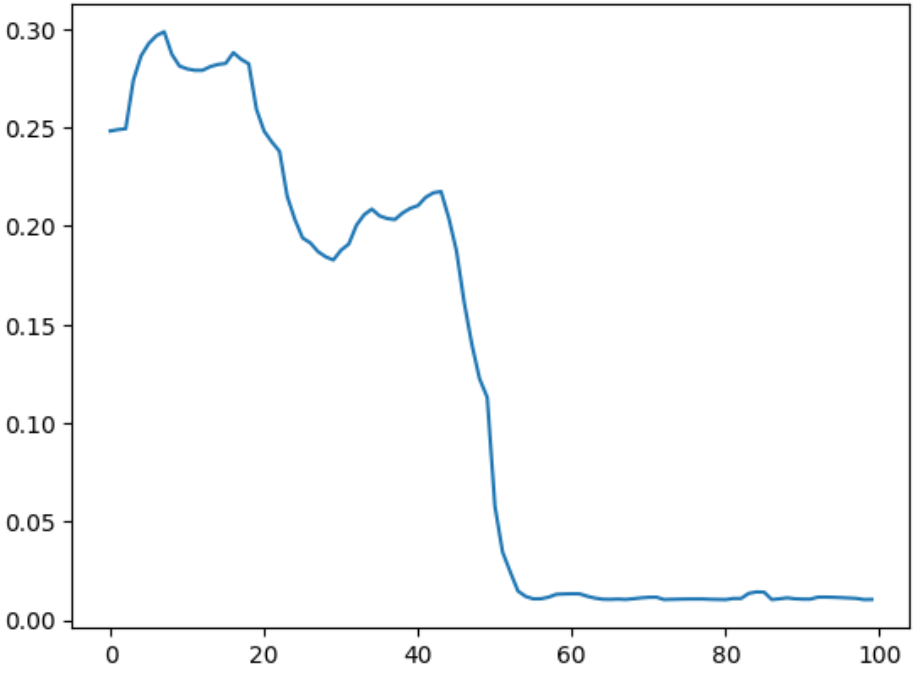
Regardless of the weights, we also note that, if the direction of the past gradients changes a lot over time, many terms in the sum partially cancel out (e.g., if we hav $g^{(5)} = -g^{(6)}$).

The overall effect is a smoother trajectory and reduced oscillatory behavior.

Let's solve the fitting problem with the Gabor function starting from $\omega = 1$, $\phi = 1$, with $\alpha = 0.01$, $100$ iterations, and a batch size $|B^{(t)}|$ of $1$ using SGD and SGDM:




SGD




SGDM

SGDM is smoother and seems to make more progress: it reaches the global minimum in a little over $50$ iterations, whereas SGD requires about $90$.

# In the fog, but with physics

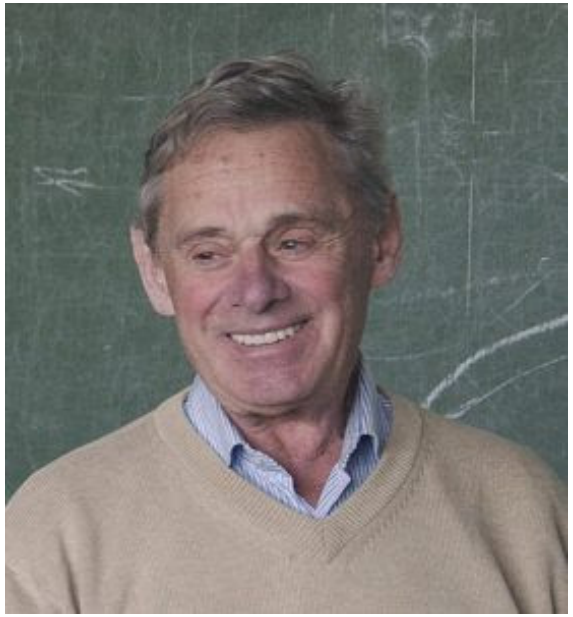
With momentum, we model the movement we make not just based on the current slope (gradient), but also on the "general" direction in which we've been moving up to now.

- $m^{(t)}$ (the momentum) is our current **velocity vector**
- $g^{(t)}$ (the gradient) is a new **force** that pushes us
- $\beta$ is a **momentum coefficient** (the higher, the more momentum we have—i.e., the higher, the more we keep moving based on our current velocity and we ignore the new force that's pushing us).

Momentum makes us more "robust" to bumps on our way down (hopefully) the foggy hill. A stone won't make us drastically change our direction if $\beta$ is high enough—but, of course, if $\beta$ it too high, then we'll then too stubbornly follow the direction we've been taking for a while, without adapting to nature of the location we currently are at.

# A historical note

The idea of momentum (sometimes called **heavy-ball**) is rather old—it's due to B. Polyak, from a 1964 article on solving functional equations.



It was popularized by D.E. Rumelhart, G.E. Hinton, and R.J. Williams's **Learning representations by back-propagating errors** paper (Nature, 1986), which is the very paper that popularized the backpropagation algorithm.

# 2. Nesterov's accelerated (stochastic) gradient descent (NAG and SNAG)

# Yuri Nesterov

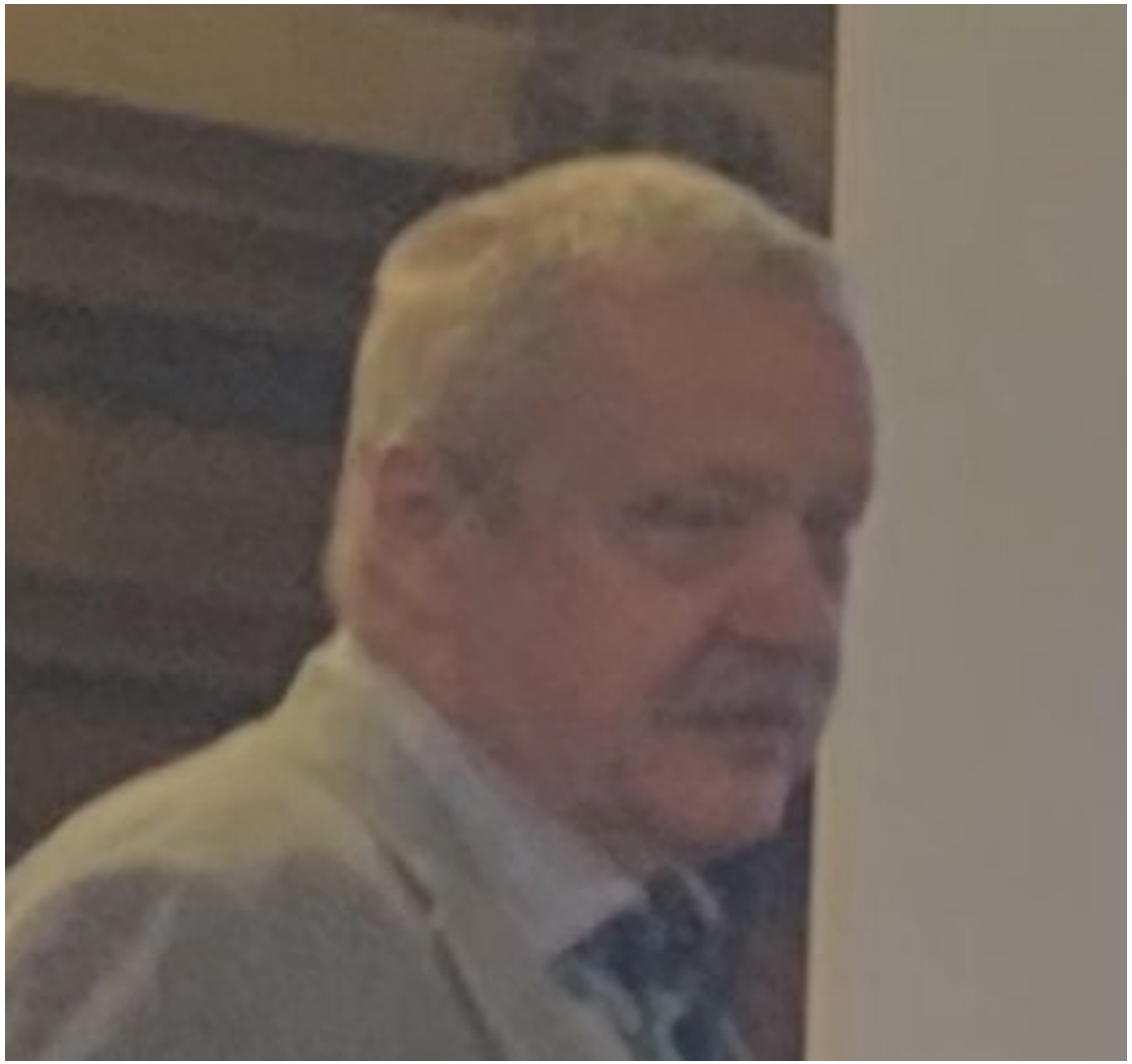
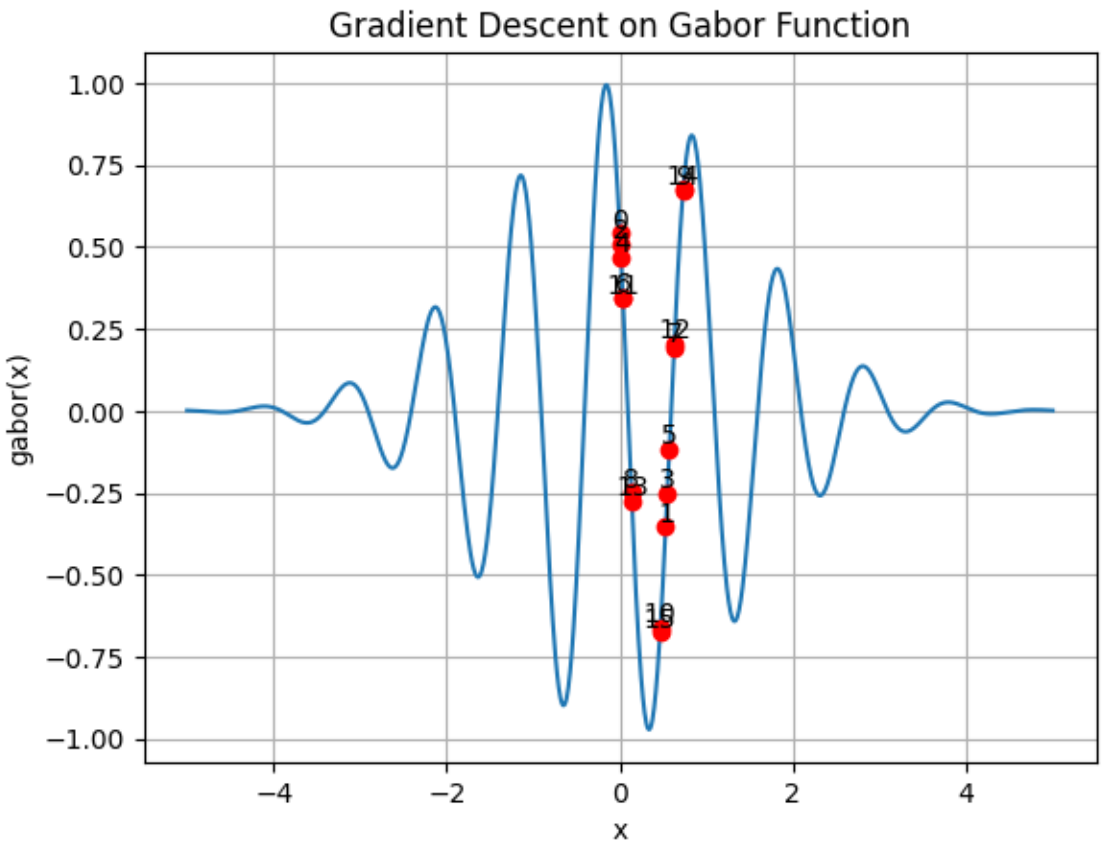


Photo taken in January 2025 in Bari at the UMI meeting on AI.

# Nesterov's acceleration: a look-ahead idea

As one may intuit, the gradient information is only accurate where it is computed.

If the slope of the loss function changes rapidly, it is likely that the gradient evaluated at step $t$ be entirely unrelated to the gradient at step $t+1$. This the source of the oscillatory behavior we observed before:



If we evaluate the gradient $g^{(t)}$ at $w^{(t)}$ and use it to move to $w^{(t+1)}$, there's a big chance (depending on the smoothness of $L$) that the gradient $g^{(t+1)}$ at $w^{(t+1)}$ will be much different from $g^{(t)}$—so different, that moving to $w^{(t+1)}$ from $w^{(t+1)}$ will, in some cases, force us to backtrack in some component of the vector of parameters.

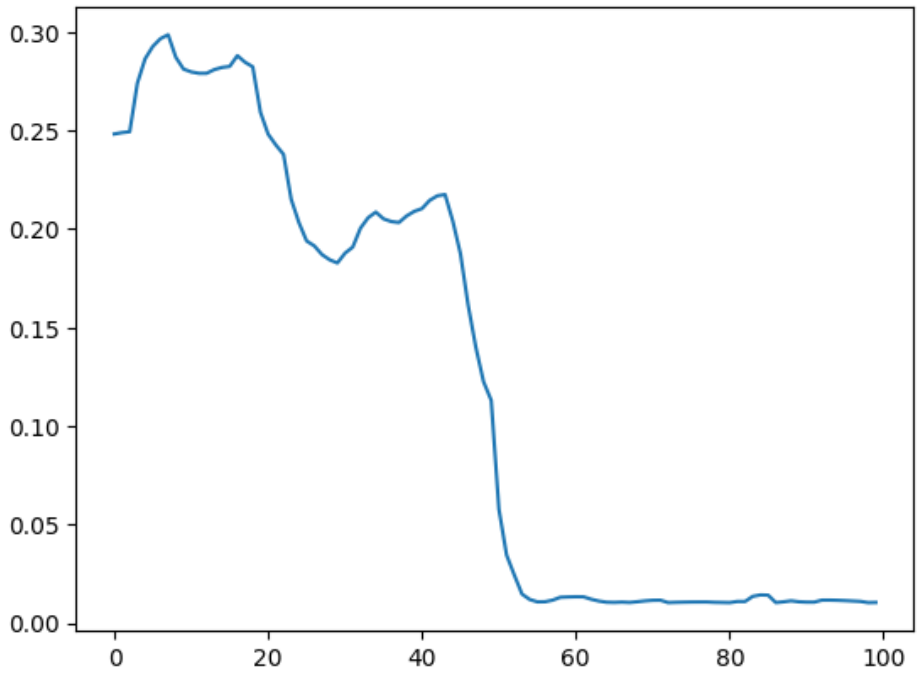
Nesterov's idea is this: instead of computing the gradient at the current location $w^{(t)}$, we evaluate it at an **estimate of the next position** $w^{(t)} - \alpha m^{(t)}$. The difference from what we did in the previous slides is in red:

$$
\begin{aligned}
g^{(t)} &= \nabla_w L_i(w^{(t)} - \alpha m^t) \\
m^{(t+1)} &:= \beta m^{(t)} + (1-\beta) \sum_{i \in B^{(t)}} g^{(t)} \\
w^{(t+1)} &:= w^{(t)} - \alpha m^{(t+1)}.
\end{aligned}
$$

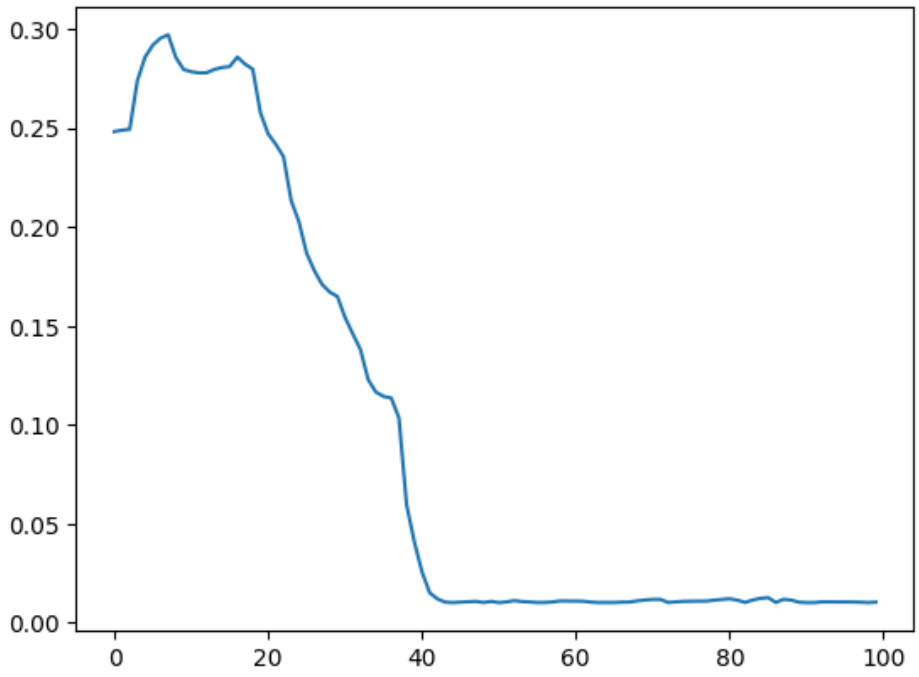
This way, we are adjusting our direction based on an estimate of where we'll be next (which coincides with where we'd be if we only relied on momentum to move).

When employing this update rule, we have the **Nesterov Accelerated GD method (NAG)** if we compute the full gradient at every step, and its stochastic counterpart the **Nesterov Accelerated SGD method (SNAG)** when the batch is not full.

Let's solve the fitting problem with the Gabor function starting from $\omega = 1$, $\phi = 1$, with $\alpha = 0.01$, $100$ iterations, and a batch size $|B^{(t)}|$ of $1$ using SGDM and SNAG:




SGDM




SNAG

SNAG appears to be even smoother than SGDM, and it converges in only $40$ iterations in this case (rather than $50$).

# Recap

SGDM and SNAG share the same update rules:

$$m^{(t+1)} := \beta m^{(t)} + (1-\beta) \sum_{i \in B^{(t)}} g^{(t)}$$
$$w^{(t+1)} := w^{(t)} - \alpha m^{(t+1)}.$$

They only differ in where $g^{(t)}$ is evaluated:

$$g^{(t)} = \nabla_w L_i(w^{(t)}) \quad g^{(t)} = \nabla_w L_i(w^{(t)} - \alpha m^t)$$

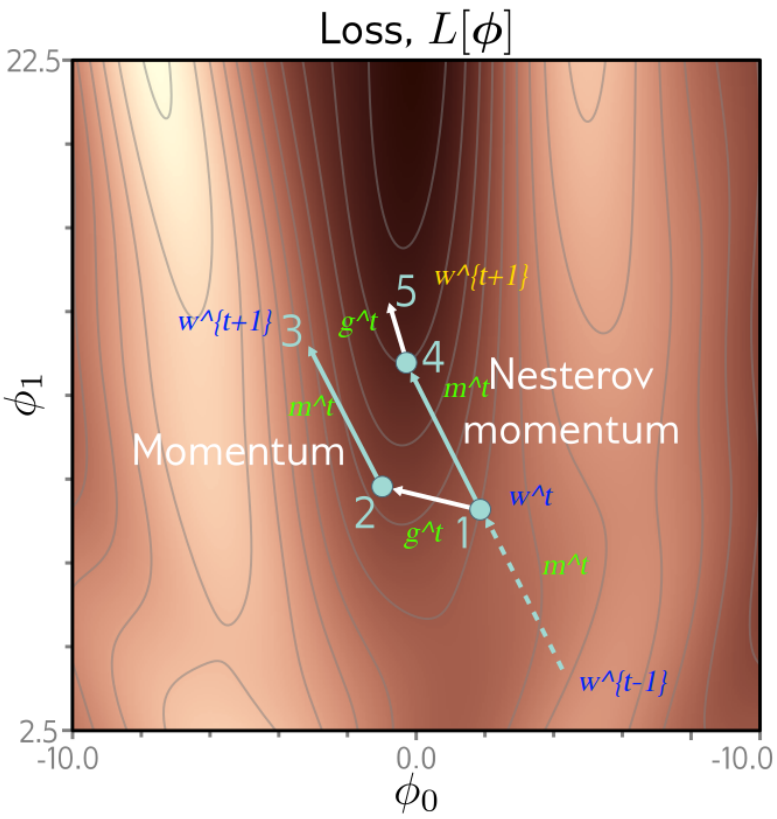
SGDM SNAG

We can think of both methods carrying out two moves at once:

- SGDM first moves in the direction of the gradient $g^{(t)}$ (evaluated at $w^{(t)}$) and then of the momentum $m^{(t)}$
- SNAG first moves in the direction of the momentum $m^{(t)}$, then evaluates the gradient $g^{(t)}$ (at $w^{(t)} - \alpha m^{(t)}$) and *only then* moves in its direction.

# Example (with $|B^{(t)}| = k$)

We arrived at $w^{(t)}$ (1) following the dashed line (in both algs., this is the momentum $m^{(t)}$ that took us to $w^{(t)}$ from $w^{(t-1)}$).




SGDM evaluates the gradient $g^{(t)}$ at $w^{(t)}$ (1), moves along it to (2), and then applies the "old" momentum $m^{(t)}$ to arrive at $w^{(t+1)}$ (3).

SNAG first applies the "old" momentum $m^{(t)}$, thus moving from $w^{(t)}$ (1) to $w^{(t)} - \alpha\beta m^{(t)}$ (4); it evaluates the gradient $g^{(t)}$ *there*, and moves along it to $w^{(t+1)}$ (5).

# 3. Adam

# Towards Adam

**Adam** is the *de facto* state-of-the-art optimization algorithm used for training (deep) neural networks.

It was proposed by D.P. Kingma and J. Ba in the 2015 paper *Adam: A Method for Stochastic Optimization*.

It is based on a few ideas, all of which boil down to the construction of rules that allow for an **adaptive learning rate**.

Let's dig into them one by one.

Published as a conference paper at ICLR 2015

# ADAM: A METHOD FOR STOCHASTIC OPTIMIZATION

**Diederik P. Kingma***
University of Amsterdam, OpenAI
dpkingma@openai.com

**Jimmy Lei Ba***
University of Toronto
jimmy@psi.utoronto.ca

## ABSTRACT

We introduce *Adam*, an algorithm for first-order gradient-based optimization of stochastic objective functions, based on adaptive estimates of lower-order moments. The method is straightforward to implement, is computationally efficient, has little memory requirements, is invariant to diagonal rescaling of the gradients, and is well suited for problems that are large in terms of data and/or parameters. The method is also appropriate for non-stationary objectives and problems with very noisy and/or sparse gradients. The hyper-parameters have intuitive interpretations and typically require little tuning. Some connections to related algorithms, on which *Adam* was inspired, are discussed. We also analyze the theoretical convergence properties of the algorithm and provide a regret bound on the convergence rate that is comparable to the best known results under the online convex optimization framework. Empirical results demonstrate that Adam works well in practice and compares favorably to other stochastic optimization methods. Finally, we discuss *AdaMax*, a variant of *Adam* based on the infinity norm.

# The motivating issue

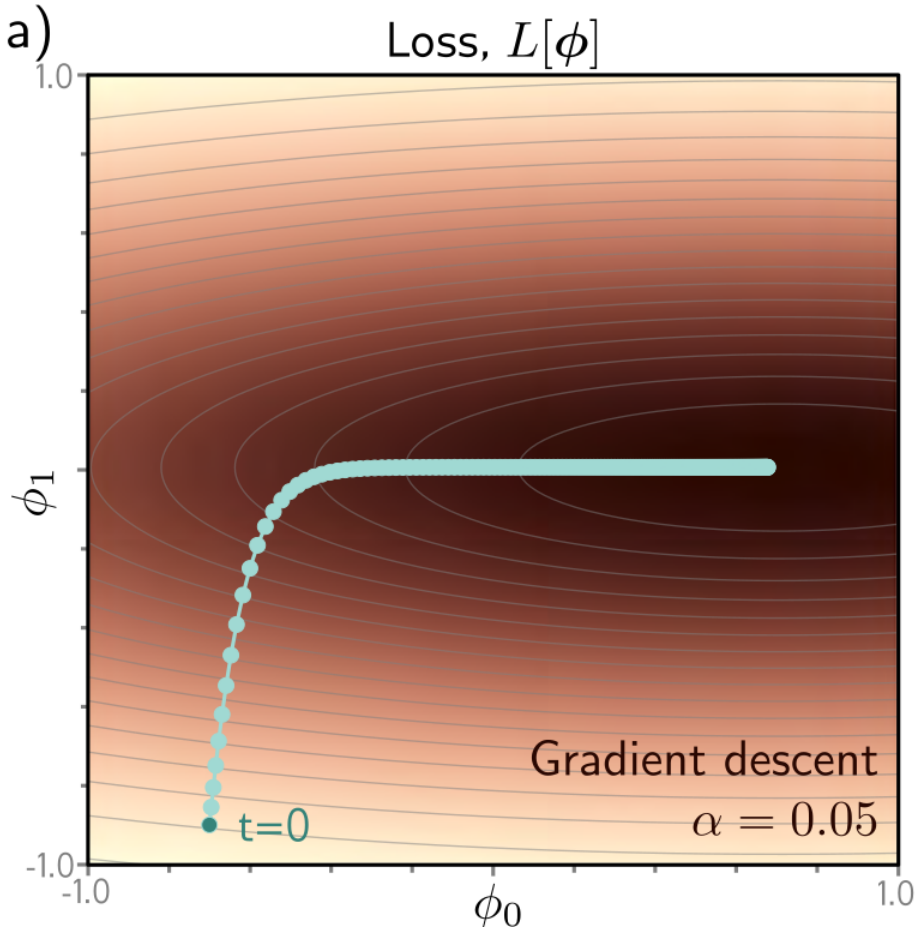
Observation: in SGD, we make large adjustments whenever the gradient is large, and small ones whenever it is small.

The learning rate mitigates this somehow, but it does so in a non-adaptive way—e.g., $\alpha = 0.5$ divides by $2$ every gradient, be it large or small.

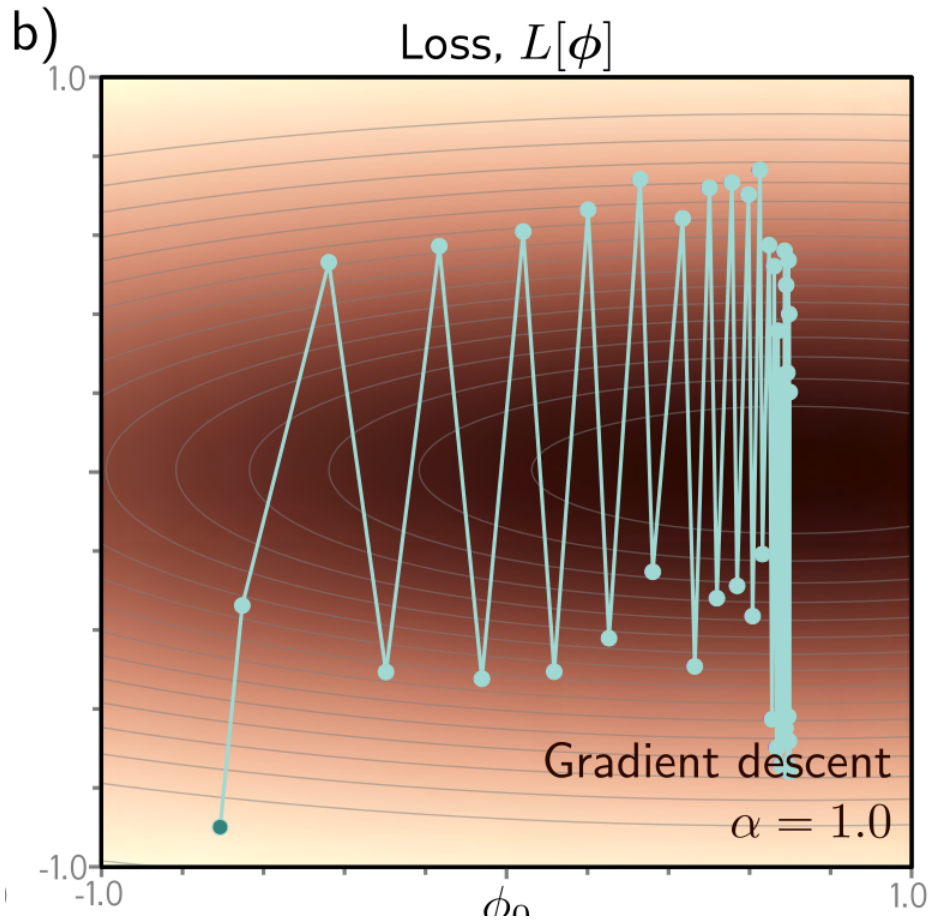
What is more, even if we were to chose $\alpha^{(t)}$ adaptively by, for instance, redefining it as $\alpha^{(t)} := \alpha \frac{1}{||g^{(t)}||_2}$ (this would normalize every gradient to unit norm), it could still be the case that the gradient of the loss is steeper in one direction than another (i.e., big in $w_2$ and small in $w_1$, which makes it impossible to choose a single learning rate that makes good progress in both directions.

Let's see a few examples.

A loss function whose gradient is steep along the vertical axis ($w_2$) but relatively flat along the horizontal one ($w_1$) (the level curves are almost flat horizontally).




GD with a learning rate that leads to good progress on the vertical axis (first part before the bend) takes a long time to reach the bottom of the valley once the algorithm starts moving purely horizontally (after the bend)—$\alpha$ is OK for $w_2$ but too small for $w_1$.




GD with a higher learning rate: we achieve good progress on the horizontal axis, but we experience an unstable behavior on the vertical axis, with multiple overshoots of what would be an ideal direction—$\alpha$ is OK for $w_1$ but too large for $w_2$.

# The 1st idea (sgnSGD)

**Idea**: let's normalize the (stochastic) gradient (no moment nor acceleration for now) component by component. Let $v$ be the componentwise-squared gradient

$$v^{(t)} := \left(g^{(t)}\right)^2,$$

where the *exponentiation is applied component-wise* (this is `v = g**2` in Python). The update rule is

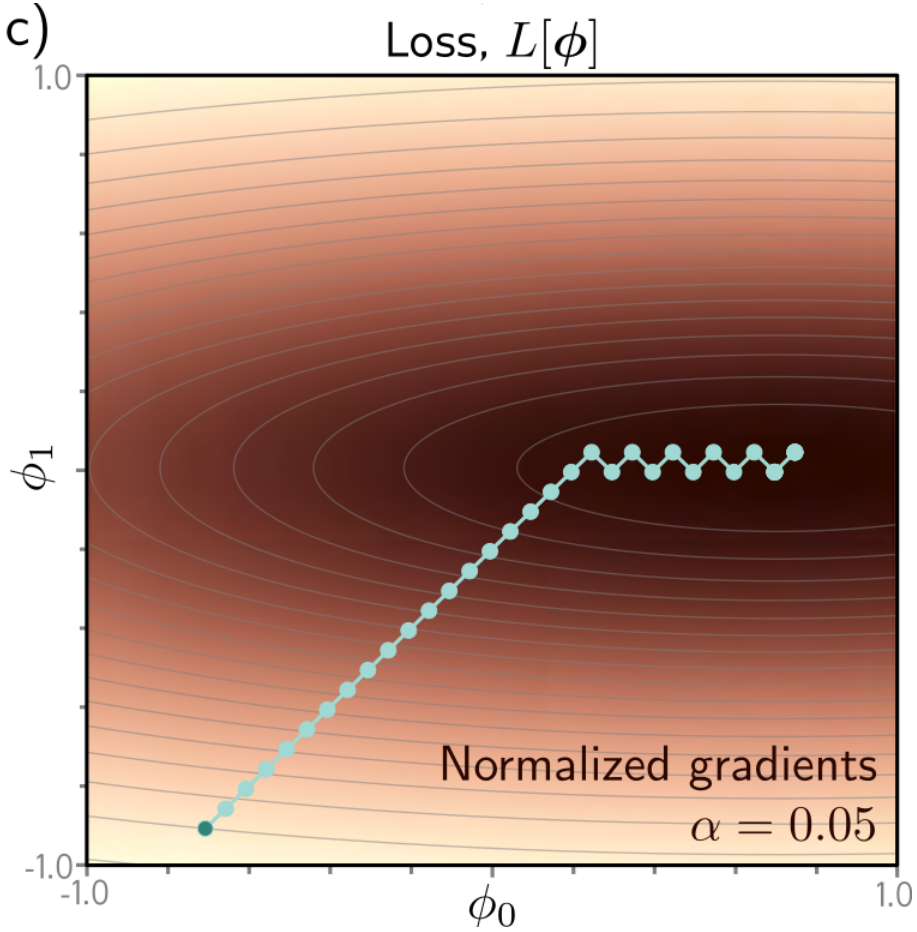
$$w^{(t+1)} := w^{(t)} - \alpha \frac{g^{(t)}}{\sqrt{v^t} + \epsilon},$$

where the *division is carried out component-wise* (this is `gnorm = g / (v**.5 + eps)` in Python) and the $+\epsilon$ terms avoids a division by $0$ when $g_j^{(t)} = 0$).

For each component $j$, $\dfrac{g_j^{(t)}}{\sqrt{(g_j^{(t)})^2} + \epsilon} = \dfrac{g_j^{(t)}}{|g_j^{(t)}| = \epsilon} \simeq \begin{cases} \operatorname{sgn}(g_j^{(t)}) & \text{if } g_j^{(t)} \neq 0 \\ 0 & \text{otherwise.} \end{cases}$

Thus, $w^{(t+1)} := w^{(t)} - \alpha \dfrac{g^{(t)}}{\sqrt{v^t} + \epsilon}$ is almost identical to $w^{(t+1)} := w^{(t)} - \alpha \operatorname{sgn}(g^{(t)})$.

Thus, with this rule we move either a fixed distance $\alpha$ or $0$ along each coordinate: hence this method is called **signum SGD (sgnSGD)**. While this may seem a rather wild method, it's not too bad. Let's see how it performs.




sgnSGD. The trajectory is rather stable (no big overshoots), but it exhibits an oscillatory behavior caused by the implicit presence of the sgn function in the update rule (the gradient can only take one of 8 directions!).

This idea (extended as we will see) is rather crucial in many NNs such as, e.g., transformers, where embeddings and attention weights have very different dynamics in the loss function.

# The 2nd idea and RMSprop

The 2nd idea is an unpublished idea that stemmed from a series of lectures by Geoffrey Hinton (Coursera Neural Networks for Machine Learning, 2018).

It boils down with combining sgnSGD with momentum, using the latter **only** to update $v^{(t)}$ using a parameter $\gamma \in [0, 1)$:

$$\begin{aligned} g^{(t)} &= \nabla_w L_i(w^{(t)}) \\ v^{(t+1)} &:= \gamma v^{(t)} + (1-\gamma) \sum_{i \in B^{(t)}} (g^{(t)})^2 \\ w^{(t+1)} &:= w^{(t)} - \alpha \frac{g^{(t)}}{\sqrt{v^{(t+1)}} + \epsilon}. \end{aligned}$$

(no momentum and no acceleration are used).

Here, we don't get $\frac{g^{(t)}}{\sqrt{(v^{(t)})^2} + \epsilon} \simeq \mathrm{sgn}(g^{(t)})$ as before because we are not dividing each component of $g^{(t)}$ by its absolute value; rather, we are dividing it by a stabilized, smoothed estimate of its magnitude in each component.

# The third idea: more momentum!

We used momentum for $v$ and, thanks to SGDM, we have an intuition that momentum works in the un-normalized case. So why not using momentum not just for $v$ but also for $g$?

Here are the updated rules:

$$
\begin{aligned}
g^{(t)} &= \nabla_w L_i(w^{(t)}) \text{ (again, no lookahead)} \\
m^{(t+1)} &:= \beta m^{(t)} + (1-\beta) \sum_{i \in B^{(t)}} g^{(t)} \\
v^{(t+1)} &:= \gamma v^{(t)} + (1-\gamma) \sum_{i \in B^{(t)}} (g^{(t)})^2 \\
w^{(t+1)} &:= w^{(t)} - \alpha \frac{m^{(t+1)}}{\sqrt{v^{(t+1)}} + \epsilon}.
\end{aligned}
$$

Notice that we are using two different hyperparameters $\beta$ and $\gamma$. Here, in essence, we have two momenta (some papers called them *moments*, harking back to stats): one for $g$ (called $m$) as in SGDM and one for its componentwise square $g^2$ (called $v$) as in RMSProp.

To the best of your lecturer's knowledge, this method has no specific name—we could call it *Adam without bias correction* (see further).

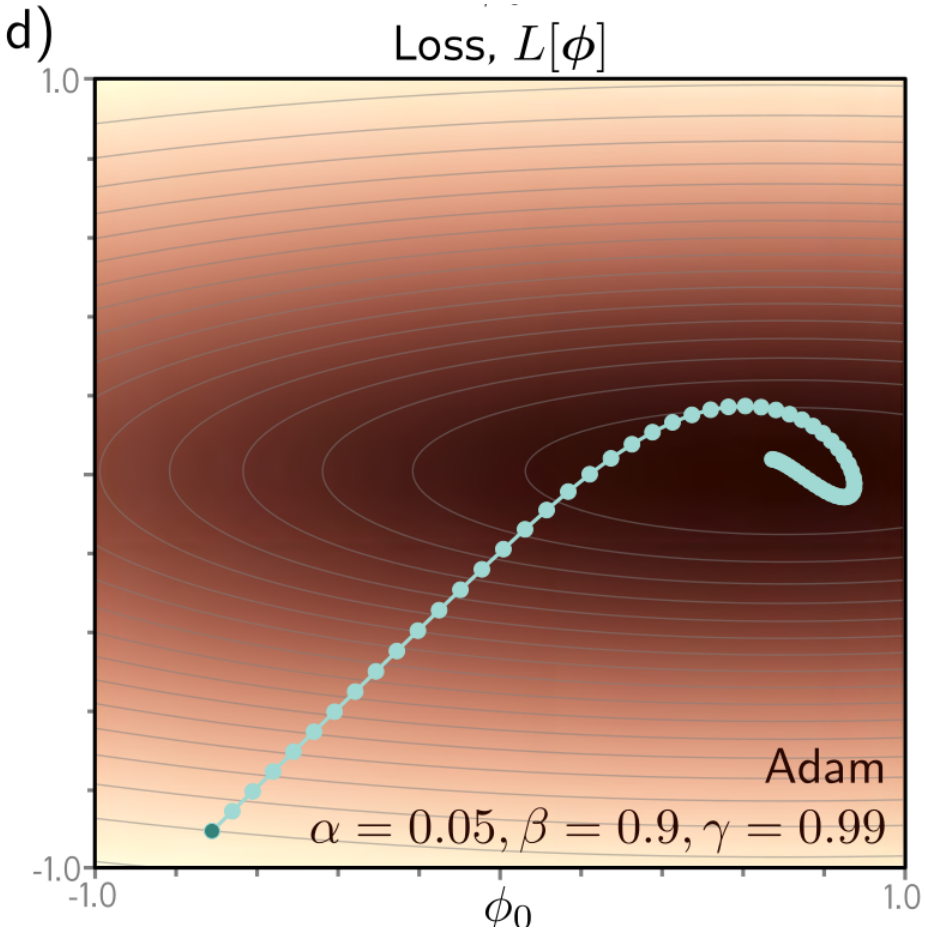
# The fourth idea: bias correction and Adam

Using the previous method (the one with two moments), at the start of the procedure we have $m^{(0)} = v^{(0)} = 0$. This leads to computing $m^{(1)} := (1-\beta) \sum_{i \in B^{(t)}} g^{(0)}$ and $v^{(1)} := (1-\gamma) \sum_{i \in B^{(t)}} (g^{(0)})^2$. If $\beta$ and $\gamma$ are large, this results in computing (and using) a small $m^{(1)}$ and a small $v^{(1)}$. Therefore, the initial updates are unfortunately **biased** towards $0$ (slowing down our initial progress!)

To avoid this, Adam employs **bias-corrected** quantities in the update rules:

$$\begin{aligned}
\hat{m}^{(t+1)} &:= \frac{m^{(t+1)}}{1-\beta^{(t+1)}} \\
\hat{v}^{(t+1)} &:= \frac{v^{(t+1)}}{1-\gamma^{(t+1)}} \\
w^{(t+1)} &:= w^{(t)} - \alpha \frac{\hat{m}^{(t+1)}}{\sqrt{\hat{v}^{(t+1)}} + \epsilon}.
\end{aligned}$$

The extra coefficients $\frac{1}{1-\beta^{(t+1)}}$ and $\frac{1}{1-\gamma^{(t+1)}}$ avoid the issue by completely canceling out $(1-\beta)$ and $(1-\gamma)$ at $t = 1$. Since $\beta, \gamma \in [0, 1)$, the extra coeffs. approach $1$ relatively fast at $t$ grows, thus only affecting the initial iterations.




Adam's trajectory: it's smooth, and it converges to the minimum.

# For the sake of completeness: here's Nadam!

As we have seen, Adam does not employ Nesterov's acceleration (the gradient is evaluated at $w^{(t)}$).

We could, if we wanted, modify Adam so that the gradient is evaluated with a lookahead as in SNAG.

Such an algorithm is usually called **Nadam**—it was proposed by T. Dozat in the ICLR 2016 paper *Incorporating Nesterov Momentum into Adam*.

https://openreview.net/forum?id=OM0jvwB8jIp57ZJjtNEZ